DS5102: Big Data Lab

Lecture-4: Optimizing MapReduce Post Lab Report

Muhamed Rizwan Mehaboob (142301026)

22th February 2026

Introduction

In this lab we studied how MapReduce performance can be improved using **Combiners**. Normally, the mapper produces many intermediate key-value pairs which are transferred to reducers during the shuffle phase, creating heavy network communication.

A combiner acts like a small reducer that runs on the mapper's output. Instead of sending every pair, it first aggregates values with the same key locally and then sends only the partial result.

Through this lab, we observed how reducing intermediate data affects the execution time and efficiency of MapReduce programs.

Task 1: WordCount

(i) Without Combiner

Python Script:

```
from mrjob.job import MRJob
from mrjob.step import MRStep

class WordCounter(MRJob):

        #Defining the MapReduce steps
        def steps(self):
                return [MRStep(mapper=self.mapper, reducer=self.reducer)]


        #Mapping
        #Reading each line from the input file
        #Splitting the line into individual words
        #Emitting each word with count 1
        def mapper(self, _, line):
                for word in line.split():
                        yield word, 1


        #Reducing
        #Receiving all counts for each word
        #Summing up the counts to get total occurrences
        def reducer(self, word, counts):
                yield word, sum(counts)


if __name__ == '__main__':
        WordCounter.run()
```

Output:

```
hadoop@machine1:~/Downloads/Lecture4/Codes$ hdfs dfs -cat /Lecture4/Output/Task1/a/part-00000 | head
-20
"1"     6834
"10"    273
"100"   567
"1000"  10
"1001"  17
"1002"  8
"1003"  8
"1004"  9
"1005"  22
"1006"  23
"1007"  47
"1008"  37
"1009"  64
"101"   140
"1010"  44
"1011"  93
"1012"  100
"1013"  38
"1014"  98
"1015"  12
```

Analysis:

Without using a combiner, every occurrence of a word produced a separate (word, 1) pair. Even if the same word appeared many times within the same mapper, all those pairs were still transferred individually to the reducer.

So if the dataset contained N words, approximately N intermediate records were shuffled across the network. The reducers then performed all the aggregation work, making the shuffle phase heavy and slower.

(ii) With Combiner

Python Script:

```
from mrjob.job import MRJob
from mrjob.step import MRStep

class WordCount(MRJob):
        #Defining the MapReduce steps
        def steps(self):
                return [MRStep(mapper=self.mapper,combiner=self.combiner,
                   reducer=self.reducer)]

        #Reading each line from the input file and plitting the line into
           individual words
        #Emitting each word with count 1
        def mapper(self, _, line):
                for word in line.split():
                        yield word, 1

        #Locally summing the word counts at mapper side
        #This reduces the number of intermediate records sent to reducer
        def combiner(self, key, counts):
                yield key, sum(counts)

        #Receiving all partial counts for each word
```

```
        #Summing them to get the final total count
        def reducer(self, key, counts):
                yield key, sum(counts)


if __name__ == '__main__':
        WordCount.run()
```

Output:

```
hadoop@machine1:~/Downloads/Lecture4/Codes$ hdfs dfs -cat /Lecture4/Output/Task1/b/part-00000 | head
-20
"1"     6834
"10"    273
"100"   567
"1000"  10
"1001"  17
"1002"  8
"1003"  8
"1004"  9
"1005"  22
"1006"  23
"1007"  47
"1008"  37
"1009"  64
"101"   140
"1010"  44
"1011"  93
"1012"  100
"1013"  38
"1014"  98
"1015"  12
```

Bandwidth Reduction:

When a combiner is added, the mapper locally counts repeated words before sending data to reducers. Instead of sending many (word, 1) pairs, it sends a single pair containing the partial count. This decreases shuffle traffic, especially for very frequent words.

Task 2: Estimating π (Using Combiner)

Python Script:

```
from mrjob.job import MRJob
from mrjob.step import MRStep

class EstimatePi(MRJob):

    #Defining the MapReduce steps
    def steps(self):
        return [MRStep(mapper=self.mapper,combiner=self.combiner,reducer=
            self.reducer)]

    #Mapping
    #Reading each line which contains x and y coordinates
    #Checking whether the point lies inside the unit circle
    #Emitting the tuple (inside_count, total_count)
    def mapper(self, _, line):
        x, y = map(float, line.strip().split())
        #If point lies inside circle (x^2 + y^2 <= 1)
```

```
        if x*x + y*y <= 1:
            yield "counts",(1,1)    #1 inside,1 total
        else:
            yield "counts",(0,1)    #0 inside,1 total


    #Combining
    #Locally summing inside points and total points
    #This reduces the number of records sent to reducer
    def combiner(self,key,values):
        inside =0
        total =0
        for i,t in values:
            inside+= i
            total +=t
        yield key, (inside,total)


    #Reducing
    #Final aggregation of inside points and total points
    #Computing the estimated value of    using Monte Carlo formula
    def reducer(self,key,values):
        inside =0
        total =0
        for i,t in values:
            inside+= i
            total+= t

        pi = 4*inside/total
        yield "Estimated Pi",pi


if __name__ == '__main__':
    EstimatePi.run()
```

Output:

```
hadoop@machine1:~/Downloads/Lecture4/Codes$ hdfs dfs -cat /Lecture4/Output/Task2/a/part-00000
"Estimated Pi"  3.13924
```

Optimization Analysis:

Each mapper generated two counters:

- ("t") for total samples
- ("i") for points inside the circle

Without a combiner, every random point produced an output record, resulting in a very large number of intermediate values. With the combiner, these counts were summed locally on each mapper and only the partial totals were sent to the reducer.

Instead of thousands of records per mapper, only a few summarized values were transferred, making the job significantly more efficient.

Task 3: Movie Rating Analysis (Using Combiner)

Python Script:

```
from mrjob.job import MRJob
from mrjob.step import MRStep

class AvgRatingPerMovie(MRJob):

    #Defining the MapReduce steps
    def steps(self):
        return [MRStep(mapper=self.mapper,combiner=self.combiner,reducer=
           self.reducer)]

    #Mapping
    #Reading each line from u.data file
    #Extracting movie_id and rating
    #Emitting movie_id as key and (rating,1) as value
    #1 represents count of one rating
    def mapper(self, _, line):
        user_id, movie_id, rating, timestamp = line.split()
        yield movie_id, (int(rating), 1)


    #Combining
    #Locally summing ratings and counts for each movie
    #This reduces the number of records sent to reducer
    def combiner(self,key, values):
        total_rating = 0
        count = 0
        for rating, c in values:
            total_rating += rating
            count += c
        yield key, (total_rating,count)


    #Reducing
    #Final aggregation of total ratings and counts
    #Computing the average rating for each movie
    def reducer(self, key,values):
        total_rating = 0
        count = 0
        for rating, c in values:
            total_rating += rating
            count += c
        yield key, total_rating/count


if __name__ == '__main__':
    AvgRatingPerMovie.run()
```

Output:

```
hadoop@machine1:~/Downloads/Lecture4/Codes$ hdfs dfs -cat /Lecture4/Output/Task3/part-00000 | head -2
0
"1"     3.8783185840707963
"10"    3.831460674157303
"100"   4.155511811023622
"1000"  3.0
"1001"  2.0
"1002"  1.875
"1003"  2.25
"1004"  3.111111111111111
"1005"  3.6818181818181817
"1006"  2.8260869565217392
"1007"  4.127659574468085
"1008"  3.3783783783783785
"1009"  3.375
"101"   3.26027397260274
"1010"  3.25
"1011"  3.225806451612903
"1012"  3.53
"1013"  2.289473684210526
"1014"  3.061224489795918
"1015"  2.8333333333333335
```

Optimization Insight:

Each rating originally produced a (movieID, (rating, 1)) pair. Since many users rate the same movie, the same key appeared repeatedly.

With a combiner, ratings for the same movie within a mapper were aggregated as:

$$(movieID, (sum_ratings, count))$$

Reducers then computed the final average using only partial totals instead of processing every individual rating. This reduced shuffle data and improved execution time.

Task 4: Matrix Multiplication

(i) One Map and Two Reducers (With Combiner)

Python Script:

```
from mrjob.job import MRJob
from mrjob.step import MRStep

class MatrixMultiply(MRJob):
    def steps(self):
        #Defining the One Mapper, One Combiner, Two Reducers Steps
        return [MRStep(mapper=self.mapper,combiner=self.combiner,
                reducer=self.reducer_partial), MRStep(reducer=self.
                   reducer_final)]

    #Mapper
    #Reading each matrix entry and emitting key-value pairs
    #so that A(i,k) and B(k,j) meet at same reducer key (i,j)
    def mapper(self, _, line):
        matrix,i, j, val = line.split()
        i,j,val = int(i),int(j),int(val)

        if matrix == 'a':
            # Emit A values for all possible result columns
```

```
            for col in range(5):
                yield (i, col), ('a',j,val)

        elif matrix == 'b':
            # Emit B values for all possible result rows
            for row in range(5):
                yield (row,j), ('b', i, val)


    #Combiner
    #Locally grouping A and B values by index k
    #Reduces duplicate entries before shuffle
    def combiner(self,key,values):
        a_vals ={}
        b_vals ={}

        for mat,k,val in values:
            if mat == 'a':
                a_vals[k] = a_vals.get(k,0) + val
            else:
                b_vals[k] = b_vals.get(k,0) + val

        for k, val in a_vals.items():
            yield key, ('a',k,val)

        for k, val in b_vals.items():
            yield key, ('b',k,val)


    #Reducer 1
    #Matching A(i,k) with B(k,j)
    #Computing partial products
    def reducer_partial(self,key,values):
        a_vals= {}
        b_vals= {}

        for mat,k,val in values:
            if mat == 'a':
                a_vals[k]= val
            else:
                b_vals[k] =val

        for k in a_vals:
            if k in b_vals:
                yield key, a_vals[k]*b_vals[k]


    #Reducer 2
    #Summing partial products to produce final C(i,j)
    def reducer_final(self, key, values):
        yield key, sum(values)


if __name__ == '__main__':
    MatrixMultiply.run()
```

Output:

```
hadoop@machine1:~/Downloads/Lecture4/Codes$ hdfs dfs -cat /Lecture4/Output/Task4/a/part-00000
[0, 0]  11878
[0, 1]  14044
[0, 2]  16031
[0, 3]  5964
[0, 4]  15874
[1, 0]  4081
[1, 1]  6914
[1, 2]  8282
[1, 3]  7479
[1, 4]  9647
[2, 0]  6844
[2, 1]  9880
[2, 2]  10636
[2, 3]  6973
[2, 4]  8873
[3, 0]  10512
[3, 1]  12037
[3, 2]  10587
[3, 3]  2934
[3, 4]  5274
[4, 0]  11182
[4, 1]  14591
[4, 2]  10954
[4, 3]  1660
[4, 4]  9981
```

Bandwidth Analysis:

Matrix multiplication produces many partial products for the same output cell. Without a combiner, every partial multiplication is sent separately to the reducers.

After adding the combiner, values corresponding to the same matrix position are summed locally at the mapper side. As a result, reducers receive fewer records and less data is transferred.

(ii) One Map and One Reducer (With Combiner)

Python Script:

```
from mrjob.job import MRJob
from mrjob.step import MRStep

class MatrixMultiply(MRJob):

    def steps(self):
        #Defining the One Mapper, One Combiner, One Reducer Step
        return [MRStep(mapper=self.mapper,combiner=self.combiner,
                reducer=self.reducer)]


    #Mapper
    #Reading each matrix entry and emitting key-value pairs
    #so that A(i,k) and B(k,j) meet at same reducer key (i,j)
    def mapper(self,_,line):
        matrix,i,j,val= line.split()
        i,j,val=int(i),int(j),int(val)
```

```
        if matrix=='a':
            #Emitting A values for all possible result columns
            for col in range(5):
                yield (i,col),('a',j,val)

        elif matrix=='b':
            #Emitting B values for all possible result rows
            for row in range(5):
                yield (row,j),('b',i,val)


    #Combiner
    #Locally grouping A and B values by index k
    #Reducing duplicate entries before shuffle
    def combiner(self,key,values):
        a_vals={}
        b_vals={}

        for mat,k,val in values:
            if mat=='a':
                a_vals[k]=a_vals.get(k,0)+val
            else:
                b_vals[k]=b_vals.get(k,0)+val

        for k,val in a_vals.items():
            yield key,('a',k,val)

        for k,val in b_vals.items():
            yield key,('b',k,val)


    #Reducer
    #Matching A(i,k) with B(k,j)
    #Computing and summing partial products directly
    def reducer(self,key,values):
        a_vals={}
        b_vals={}

        for mat,k,val in values:
            if mat=='a':
                a_vals[k]=val
            else:
                b_vals[k]=val

        total=0
        for k in a_vals:
            if k in b_vals:
                total+=a_vals[k]*b_vals[k]

        yield key,total


if __name__=='__main__':
    MatrixMultiply.run()
```

Output:

```
hadoop@machine1:~/Downloads/Lecture4/Codes$ hdfs dfs -cat /Lecture4/Output/Task4/b/part-00000
[0, 0]  11878
[0, 1]  14044
[0, 2]  16031
[0, 3]  5964
[0, 4]  15874
[1, 0]  4081
[1, 1]  6914
[1, 2]  8282
[1, 3]  7479
[1, 4]  9647
[2, 0]  6844
[2, 1]  9880
[2, 2]  10636
[2, 3]  6973
[2, 4]  8873
[3, 0]  10512
[3, 1]  12037
[3, 2]  10587
[3, 3]  2934
[3, 4]  5274
[4, 0]  11182
[4, 1]  14591
[4, 2]  10954
[4, 3]  1660
[4, 4]  9981
```

Observation:

Even with a single reducer, the combiner still improves performance because duplicate intermediate values are reduced before the shuffle phase. The final computation still happens in one reducer, but unnecessary communication is avoided.

2×2 Matrix Theoretical Analysis

Consider two 2×2 matrices. Each output element is calculated using two multiplications and one addition.

In MapReduce, mappers produce partial products for each output position. Without a combiner, every partial value is sent separately to the reducer, increasing shuffle data.

With a combiner, partial products for the same position are summed locally before being transferred. The reducer then performs only the final addition.

As matrix size grows, the number of partial products per element increases, so the reduction in intermediate data and the advantage of using combiners becomes more noticeable.

Conclusion

In this lab we observed that combiners mainly optimize the shuffle phase of MapReduce by performing partial aggregation at the mapper side.

The improvement was clear in problems with repeated keys such as WordCount and Movie Rating analysis. In π estimation, the number of intermediate counter updates decreased, and in matrix multiplication the number of partial products sent to reducers was reduced.

The final output remains unchanged, but communication overhead becomes much smaller. Therefore, combiners are especially useful for aggregation-heavy MapReduce tasks.